```
$handle = fopen('php://memory', 'r+');
fputs($handle, 'Live long and prosper.');
rewind($handle);
print stream_get_contents($handle);
fclose($handle);
```

```
Output:
Live long and prosper.
```

This time we're using the `php://memory` wrapper; bytes written to that handle (using the function `fputs()`) are stored in memory and are only accessible via the `$handle` stream resource. Part of the way a PHP stream handle works is that it maintains a pointer to a specific byte position in that stream, and after data is appended, the pointer is usually at the end of the stream. The call to `fputs()` has moved the pointer to the end, so we use `rewind()` to move the pointer back to the start, which then allows `stream_get_contents()` to consume all remaining bytes from the current pointer position. We'll have a look at some more `stream_*` functions in a moment, but first there's a special case I'd like to highlight.

The `php://filter` wrapper allows you to push stream data through various filters, which allows the data to be transformed before use in the program. For example, the data could be treated as text and converted to upper or lower case, or it could be handled as a `zlib` compressed stream and decompressed by the filter. To see a list of available filters for your installation, use the following function:

```
print_r(stream_get_filters());
```

```
Output:
Array
(
    [0] => zlib.*
    [1] => convert.iconv.*
    [2] => string.rot13
    [3] => string.toupper
    [4] => string.tolower
    ...
```